

LMetric*: 大道至简——乘法或许就是 LLM 请求调度所需的一切

Dingyan Zhang^{†1}, Jinbo Han^{†1}, Kaixi Zhang^{†1}, Xingda Wei^{✉1}, Sijie Shen², Chenguang Fang²,
Wenyuan Yu², Jingren Zhou², Rong Chen¹

¹上海交通大学并行与分布式系统研究所 ² 阿里巴巴集团

摘要

高质量的 LLM 请求调度需要同时实现两个关键目标：一是被路由到的实例是否拥有可加速请求执行的 KV-Cache，二是不同实例之间的工作负载是否保持均衡。同时满足这两个目标并不容易，因为追求其中一个目标时，往往会削弱另一个目标。现有方法通常采用各种组合器（例如线性组合），将分别反映这两个目标的指标合成为一个调度分数；但这类方法较为复杂，要么需要大量面向具体工作负载的超参数调优，要么需要开发同时感知模型与硬件的模拟器，而且即便如此仍可能得到次优性能。

本文表明，只需将两个经过精心选择的指标相乘作为调度分数——一个用于感知 KVCache（若请求被路由到某实例时需要新生成的预填充 token 数），另一个用于感知负载均衡（实例当前的批大小）——就能在无需任何超参数调优的情况下，同时较好地实现这两个目标。其核心思想是：乘法分数以一种类似线性组合的方式同时考虑两个目标，并且在实例间比较时具有一个很好的性质，即原本的超参数会在比较过程中被抵消，因此我们无需调参来寻找最佳参数。这两个指标的选择来自我们对 LLM 特性的分析；大量实验表明，与 vLLM-v1 和某生产环境调度器相比，这种简单方法在覆盖聊天机器人、API 调用和代码智能体等真实工作负载上，可将 TTFT 分别降低 92% 和 52%，并将 TPOT 分别降低 21% 和 20%。我们还从数学上推导了乘法可能失效的条件，并发现这类条件在实践中极为罕见，而且可以事先被检测出来（并加以缓解）。

1 引言

本文研究如何将 LLM 请求高效路由到由服务实例组成的集群——实例是模型部署的最小单元。LLM 服务已成为现代社会的关键基础设施，而 LLM 提供商通常会部署实例集群来提供服务；每个集群中都包含一个全局调度器，负责将进入的请求路由到其管理的实例上 [32, 42, 43, 19, 27, 4, 37]。当实例接收到一个请求后，会分两个阶段生成结果 token：预填充（P）阶段生成第一个结果 token，其服务质量由首 token 时间（time-to-first-token, TTFT）衡量；随后，解码（D）阶段以流式方式生成剩余 token，其服务质量由每输出 token 时间（time-per-output-token, TPOT）衡量。

*名称 LMetric 中的 **L** 和 **M** 分别对应 Large Model 与 metric；同时它也致敬 Lyapunov——其稳定性理论部分启发了我们的乘法打分方法——以及 Markov——他对排队论作出了奠基性贡献。

[†]大部分工作完成于阿里巴巴集团实习期间。

[✉]Xingda Wei 为通讯作者（wxdwfc@sjtu.edu.cn）。

在集群级 LLM 服务中，提供有效的调度策略至关重要，因为与传统请求路由类似 [12, 45, 17]，更好的放置决策能够借助更优的实例间负载均衡等因素显著降低整体请求时延（更低的 TTFT 和 TPOT）。低时延服务对于当前的交互式应用尤其关键，例如 ChatGPT [29] 和各类 copilot [16]，因为它直接关系到能否满足用户预期 [7, 46, 15]。此外，近期的 agentic 工作负载还会通过执行计算程序快速消耗 token [6, 18, 28]。

实现良好的 LLM 特定调度策略并不简单。首先，只考虑不同实例之间的负载均衡——这是最近的最先进服务系统 vLLM [37] 以及传统请求路由所采用的方法——是不够的。原因在于，由于 KVCache——已处理 token 的中间上下文——的存在，不同实例处理同一请求所需的计算量可能不同 (§4.2)。具体而言，如果某个新到请求的（部分）输入 token 命中某实例中缓存的 KVCache，该实例就可以跳过为这些命中 token 生成对应 KVCache 的计算，从而加速后续的预填充与解码阶段。然而，只把 KVCache 感知指标纳入调度决策（例如请求被路由到某实例时的 KVCache 命中率）同样不够，因为这会让请求偏向那些有 KVCache 命中的实例，从而损害跨实例的负载均衡 (§4.3)。

为了在这两个目标——KVCache 感知与负载均衡——之间取得平衡，当前主要有三类不同的组合策略。第一类是线性组合策略（即加权求和）[23, 27]，它将分别对应这两个目标的指标组合成一个调度分数 (§4.4)。这是最简单、也最流行的一类策略，已被当前研究以及全球顶级 LLM 服务提供商之一（Bailian）采用。然而，线性组合需要复杂的、面向具体工作负载的超参数调优，才能同时兼顾两个目标。此外，静态调好的超参数在工作负载动态变化时往往又会变成次优 (§4.4)。

第二类是基于过滤的策略：它先过滤掉那些被怀疑负载失衡的实例，再在剩余实例中选择 KVCache 命中最多的实例 [3]。这类方法仍然需要不平凡的、面向具体工作负载的阈值调优来决定过滤门槛。更糟的是，它在调度上偏向负载均衡，因此无法充分利用 KVCache 缓存 (§4.5)。

最后，基于模拟的策略 [32, 19, 47, 10] 会先用模拟器预测将请求路由到各实例后的预期时延，再把该时延作为调度分数。模拟器依据当前指标（例如 KVCache 内容和排队请求）来估计时延，因此它可以被视为对其他策略所用指标的一种高阶组合。然而，这类策略的有效性高度依赖模拟器的准确性，而要做到这一点需要复杂的、面向每种模型、每种硬件以及每种部署方式的开发工作。否则，不准确的模拟就会导致糟糕的调度性能 (§4.6)。即便模拟器足够准确，在某些情况下它的性能

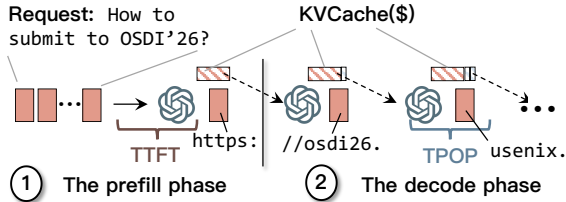


图 1: 使用 LLM 生成输出 token 的示意图, 以及两个性能指标: 首 token 时间 (TTFT) 和每输出 token 时间 (TPOT)。

也仍可能无法与其他候选方法媲美。

本文表明, 将一个用于 KVCache 感知的指标与一个用于负载均衡的指标相乘, 并把结果作为调度分数, 可以在不依赖复杂超参数调优或任何模拟器的前提下, 有效结合这两个目标。其核心思想是: 用乘法替代线性组合中的加法后, 超参数会在不同实例之间的分数比较过程中被抵消。因此, 这个分数能够保留与线性组合相似的趋势, 却不再需要任何超参数调优 (§5)。

为了让这种简单乘法在实践中真正有效, 我们发现精心选择指标十分关键 (§5.1)。例如, 将“请求被路由到某实例时、考虑 KVCache 命中后的新增预填充 token 数”作为 KVCache 感知指标, 比使用 KVCache 命中率更好。此外, 使用实例当前的批大小作为负载均衡指标, 也优于使用实例上排队总 token 数 (包括解码上下文 token)。最后, 乘法在一些罕见情况下确实会失效; 因此我们从数学上推导了这些情形的大致条件。基于这些条件, 我们发现它们在实践中极为罕见, 而且可以通过两阶段方法检测出来 (§5.2)。一旦检测到, 我们还可以退化回仅考虑负载均衡的策略来缓解问题。

我们将该方法与多种最先进方法进行了比较, 包括 vLLM [37]、AI-Dynamo [27]、LLM-D [19], 以及 Bailian——全球最大的 LLM 提供商之一——所使用的方案, 并在覆盖聊天机器人、API 调用和代码智能体的真实 LLM 服务工作负载上进行了评估 (§4.1)。在流行模型 (同时涵盖稠密模型与 MoE 架构) 上的评测表明, 在最多 16 张 H20 GPU 的集群上, 我们的方法具有明显优势。

我们将在论文发表时开源全部代码, 包括所有基准测试工具以及开放的轨迹数据。

讨论: PD 同置与 PD 解耦。 本文假设预填充与解码请求都由同一个实例处理, 这种设置称为 PD-colocation。虽然也存在预填充与解码由不同实例分别处理的部署方式, 即 PD-disaggregation [30, 46], 但 PD 同置在实践中仍被采用 [42], 因为它更容易维护 (无需管理实例角色), 也不依赖实例间的高速网络。尽管 Bailian 大量部署了 PD 解耦服务, 仍有一些服务依赖 PD 同置部署。面向 PD 解耦服务的高效调度超出了本文的讨论范围。

2 LLM 服务与调度

2.1 背景: 在集群中调度 LLM 请求

使用 LLM 处理请求 (图 1)。 LLM 以自回归方式生成 token, 整个过程分为两个步骤: ①: 在预填充阶段, 输入 token 被送入模型, 模型处理这些 token 并生成第一

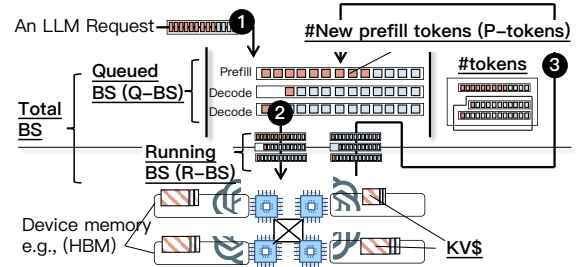


图 2: LLM 服务实例如何处理请求的系统视图, 以及全局调度器可以直接采集的一些系统指标。每个指标的详细含义会在首次使用时说明。BS 是 batch size (批大小) 的缩写。

个输出 token。之后, LLM 进入解码阶段 (②), 逐个生成后续 token。每次生成都需要所有已生成 token 以及输入 token 的上下文。为了加速生成, 这些已处理上下文会以张量形式物化并存储在 GPU 显存中, 通常称为键值缓存 (KVCache)。因此, 解码阶段的计算量显著小于预填充阶段, 因为无需再为输入 token 生成 KVCache。

从 LLM 服务提供商的角度看, 与服务质量相关的两个关键性能指标是: 首 token 时间 (TTFT) 和每输出 token 时间 (TPOT)。TTFT 很重要, 因为它直接决定了 LLM 服务对用户请求的响应速度; 而 TPOT 不仅影响后续交互的流畅度, 也影响整个请求完成所需的总时间。

服务实例与 KVCache 缓存 (图 2)。 在 LLM 服务系统中, 实例是处理请求的最小单元, 并承载一份完整的 LLM 参数副本。如果模型太大而无法装入单张 GPU 显存, 一个实例也可以由多张 GPU 组成。无论实例由单 GPU 还是多 GPU 构成, 整体服务流程都是相同的: 当实例接收到请求后, 请求会被压入该实例的队列中, 队列里存放着等待在该实例上执行的请求 (预填充或解码, ①)。当 GPU 可用时, 所有排队请求会被组成一个批次, 并结合 chunked prefill [2] 在 GPU 上高效执行 (②)。执行完成后, 实例会检查当前请求是否还需要进一步解码; 如果需要, 请求会被重新入队 (③)。

一个值得注意的特点是, 该服务是有状态的: 即使请求已经完成生成, 其 KVCache 仍会保留在 GPU 或 CPU 内存中 (即 KVCache cache [32, 14, 39])。这样做的好处在于, 如果未来某个请求被路由到某实例, 并且它与此前缓存的 KVCache 存在前缀匹配, 那么该实例就可以跳过对匹配前缀 token 的计算, 从而显著降低计算成本。在图 2 的示例中, 蓝色标出的 token 若发生 KVCache 命中, 就可以跳过对应计算。

集群中的请求调度 (图 3)。 为了处理大量请求, LLM 服务提供商通常会部署由多个实例组成的模型服务集群。每个集群都配有专门的全局调度器, 负责将请求路由到其管理的实例上, 如图 (a) 所示。调度器通过执行某种调度策略来决定每个请求的目标实例, 而这正是本文关注的重点。需要注意的是, 服务提供商也可能为同一个模型部署多个集群, 以增强可靠性与地理亲和性; 而跨集群路由不在本文讨论范围内。

从高层看, 现有所有调度策略都可以视为一个三步过

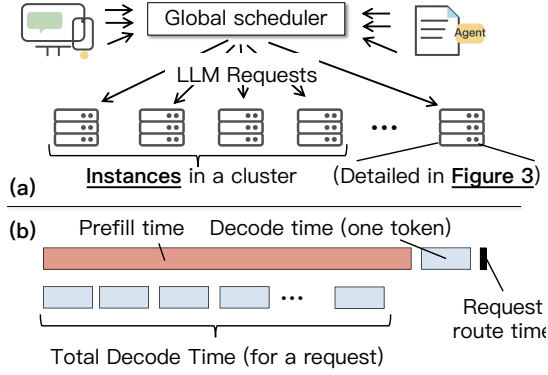


图 3: (a) 集群式 LLM 服务系统的系统视图；以及 (b) 单个请求的服务时间与路由开销的对比。

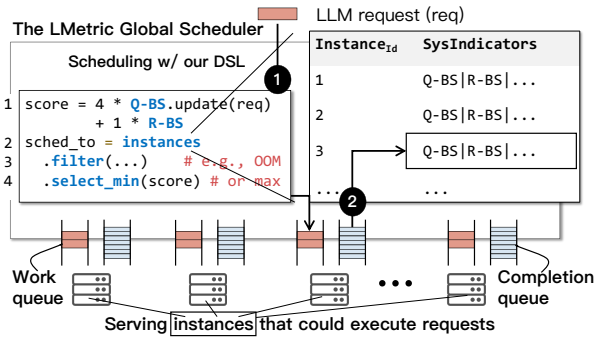


图 4: LMetric 指标工厂的系统架构，以及其面向调度算法的编程模型。

程：调度器首先（可选地）过滤掉部分实例，随后按照某种偏好顺序对实例进行打分，最后将请求路由到得分最优的实例。这个分数建立在从各实例采集到的指标之上，我们将在后续分析中详细介绍这些指标（§4）。

3 分析框架

为了以统一、公平的方式系统分析不同调度策略，我们实现了一个灵活的 LLM 调度分析框架——LMetric。推动 LMetric 设计的原因有两点：(1) 现有策略分散在不同服务系统的具体实现中（既包括开源实现，也包括闭源实现），因此很难进行真正可比的横向分析。例如，AIBrix [3] 中 vLLM 策略的 Go 实现，由于后来被确认的性能缺陷 [38]，比 vLLM 的 Python 实现 [37] 快 $6.2\times$ 。与此同时，我们基于 Rust 的实现，在使用相同 vLLM 策略的情况下，又比 AIBrix 快 $1.2\times$ 。(2) 统一框架还能简化新策略的实现与比较：尽管细节复杂，现有所有策略本质上都可以归结为依据实例指标计算调度分数。通过提供统一的指标工厂，我们的框架只需几行代码就可以探索新的策略（下文将介绍）。

指标工厂。 LMetric 是一个使用 Rust 实现的独立推理路由，可以与任意 LLM 服务引擎协同工作。如图 4 所示，其核心组件是一个指标工厂：它能够自动收集并在需要时计算不同调度策略所需的指标。选择 Rust 使得 LMetric 具备极高的效率与鲁棒性。为了获得高可扩展性，指标收集被搭载在实例返回响应这一过程中。例如，

LMetric 会与每个 vLLM 实例维持长连接；当 vLLM 返回响应时，LMetric 会从响应头中提取所需指标，并更新指标工厂中对应的条目。

编程模型。 LMetric 提供了一个简单的 API 来实现不同调度策略。具体而言，所有策略本质上都是先为每个实例计算一个分数，再将请求路由到分数最优（例如最小或最大）的实例。我们的 API 允许开发者基于指标工厂支持的“每实例符号指标”，用少量表达式定义分数函数。图 4 中第 1 行给出了 vLLM [37] 所采用的一个具体策略：它将 Q-BS（queued batch size，即实例队列中排队请求的数量）与 R-BS（实例上正在运行的请求数量）做加权求和，作为调度分数。在定义好分数函数之后，当系统需要决定新请求应被路由到哪里时（第 4 行），LMetric 会先并行地从指标工厂中读取并计算各实例的具体指标值，再推导出所有实例的分数，最后把请求路由到分数最小的实例。

4 LLM 请求调度的刻画

4.1 刻画方法

测试床与实例配置。 除非特别说明，本文所有实验都在一个包含 16 张 NVIDIA H20 GPU 的测试床上进行——该硬件与 Bailian 用于承载 LLM 服务的硬件相近。每张 GPU 具有 96 GB HBM，足以承载市场上常见的模型 [42]。路由器部署在一台高端 CPU 服务器上，配备 160 个 Intel Xeon 核心和 1 TB DRAM。我们的实例运行最新的 vLLM-v1 (vLLM) [37]——当前最先进的 LLM 服务引擎之一，并包含 chunked prefill [2] 与快速 GPU kernel [13] 等最新优化。

模型。 我们选择了具有代表性的、且在市场上广泛使用的 LLM 模型，覆盖不同架构，包括稠密模型 (Qwen2-7B) 和混合专家模型 (MoE, Qwen3-30B) [42]。

工作负载。 我们在真实世界的 LLM 服务轨迹上开展分析，这些轨迹既包括开源数据，也包括来自 Bailian 的内部采样，并覆盖了常见的 LLM 应用：ChatBot (Qwen) 和 Agent (Qwen) [9] 是阿里云开源的轨迹，分别采集自类似 ChatGPT 的聊天机器人服务以及 LLM API 调用型智能体服务 [6, 18, 28]；Coder 收集了 Bailian 某代码 copilot 服务在 2025 年 11 月某一天发往专用集群的请求；而 ToolAgent (Kimi) [26] 是另一份来自 Kimi 的开源轨迹，同样采集自智能体服务。除 ToolAgent (Kimi) 外，其余轨迹都来自单个由一个全局路由器管理的集群；ToolAgent (Kimi) 未说明具体采集方式。

我们所分析的工作负载具有代表性——不仅因为场景覆盖面广，还因为每条轨迹都保留了评估 LLM 调度策略所需的关键特征。具体来说，所有请求都包含请求内容的（哈希后）信息以及请求发起时间戳，这对于评估 KVCache 感知调度对全局调度的影响至关重要（见 §4.2）。其他流行数据集，如 AzureLLMTrace [8] 或 BurstGPT [41]，并不提供这类内容信息。需要注意的是，即使轨迹中只保存了哈希后的内容，我们仍然可以根据哈希值重建输入，从而以与原始轨迹完全一致的行为进行回放；此外，无论 EOS token 如何，各实例执行的解

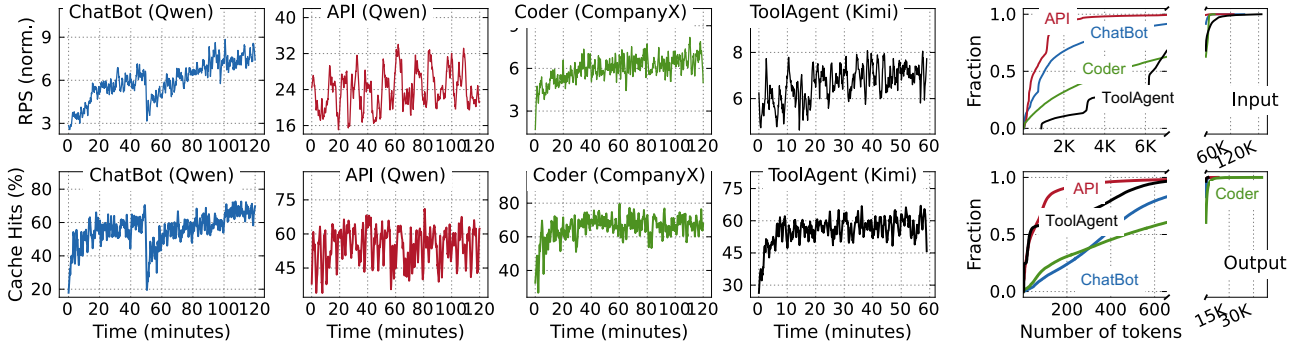


图 5: 本文研究的轨迹, 覆盖了支撑 LLM 服务的主要场景。

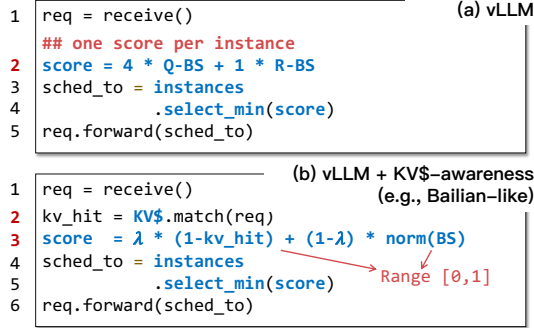


图 6: (a) vLLM 的调度分数; 以及 (b) 在其分数中加入 KVCache 感知后的 LLM 调度方式。需要注意的是, 两个指标的线性组合只有一个自由度 (权重 λ)。

码次数都与轨迹中记录的一致 [40]。

图 5 可视化了这些轨迹的关键特征, 包括请求到达率、输入与输出 token 数, 以及假设 KVCache 空间无限时的 KVCache 命中率。由于保密要求, **Coder** 轨迹中的请求到达率经过了归一化处理。在所有这些轨迹上, 我们都观察到: 在给定的服务时间区间 (例如 1 小时) 内, 请求到达率和 KVCache 命中率整体较为稳定, 仅存在少量短时波动。此外, 尽管不同轨迹的输入和输出 token 数存在差异, 但除了极少数离群点之外, 它们通常都不算特别大。

轨迹缩放。 由于这些轨迹最初采集自与我们测试床规模不同的集群, 我们参考既有工作 [25, 5, 20, 44, 34], 按照测试床能力对轨迹进行缩放。除非特别说明, 我们将平均请求到达率缩放到通过离线分析得到的测试床最大处理速率的一半。这与 Bailian 的实际服务配置相近, 因为如果到达率逼近服务能力, Bailian 常见的做法是将请求重定向到另一个负载较低的集群, 或者在非关键场景 (如 ChatAPP) 下直接拒绝部分请求 [32]。否则, 由于排队, 大量请求的服务级目标 (SLO) 将无法实现 [31]。我们在 §6 的端到端分析中还进一步测量了不同请求到达率对调度性能的影响, 结果同样保持一致。

4.2 仅靠负载均衡对 LLM 并不够

起点案例: vLLM 策略 [37]。 我们的研究从 vLLM [37] 采用的默认全局调度策略开始——它是工业界与学术

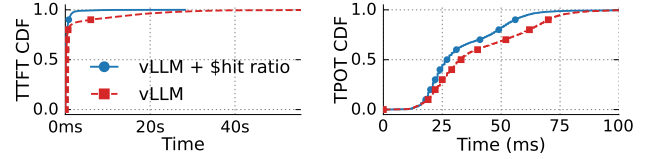


图 7: 在 Qwen3-30B 模型与 ChatBot 轨迹上, vLLM 与 KV cache 感知调度的性能对比。其他工作负载和模型也呈现类似趋势。

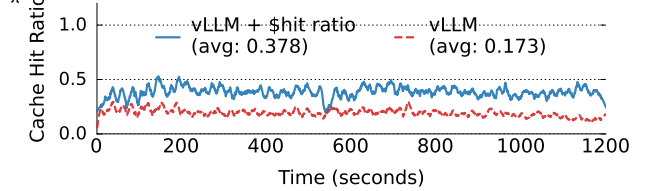


图 8: Qwen3-30B 模型在 ChatBot 轨迹上的 KVCache 命中率对比: vLLM vs. KVCache 感知调度。其他工作负载和模型也类似。

界都广泛使用的流行开源 LLM 服务引擎。图 6 (a) 展示了其调度方法: 它使用每个实例的批大小作为路由分数的指标。本质上, 它是经典“最短队列优先” (join-the-shortest-queue, JSQ) 负载均衡策略在 LLM 场景下的一个变体: 对每个实例而言, 批大小同时包含正在该实例上运行的请求数 (R-BS) 和在实例队列中等待的请求数 (Q-BS)。

为 vLLM 加入 KVCache 感知 (Bailian)。 虽然 JSQ 尝试在实例间均衡工作负载, 但它并不知道新请求的 KV-Cache 状态。而事实上, 将请求路由到具有更高 KVCache 命中率的实例, 可以显著降低请求时延。图 6 (b) 展示了一种由 Bailian 及其他工作 [27, 4] 采用的简单扩展方式, 用来让请求调度具备 KVCache 感知能力: 它在分数函数中加入了一个 KVCache 指标——一个估计当请求被路由到某实例时 KVCache 命中率的比率。需要注意, KV 是表示每实例 KVCache 哈希映射的符号值。这里有两点值得说明: (1) 负载均衡指标 (批大小) 需要被归一化到 $[0, 1]$, 以匹配命中率的量纲, 否则二者无法直接相加; (2) 本节分析假设线性组合系数 (λ) 固定, 而下一节将讨论如何设定这些系数。

正如图 7 所示, 在仅做负载均衡的策略中加入 KV-Cache 感知后, 平均 TTFT 提升了 84%, 平均 TPOT 提升了 17%。这符合预期, 因为它提高了图 8 所示的 KV-

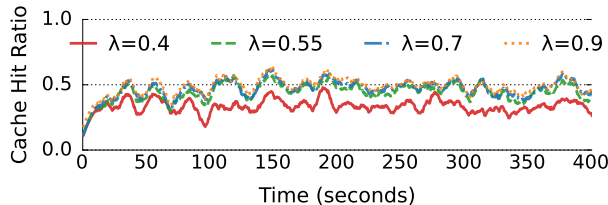


图 9: 在 Qwen3-30B 模型与 ChatBot 轨迹上, 改变 KVCACHE 感知权重时, 策略 (图 6 (b)) 的 KVCACHE 命中率对比。

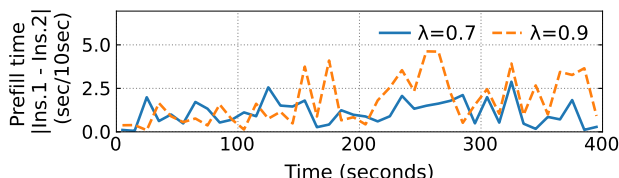


图 10: 在 ChatBot 轨迹与 Qwen3-30B 模型上, 使用两种不同线性组合权重运行时, 两台实例之间工作负载失衡情况的剖面图。图中报告的指标是在每个 10 秒时间窗内, 两台实例 (Inst.) 已服务预填充时间的绝对差值。

Cache 命中率。有趣的是, 尽管更高的 KVCACHE 命中率乍看之下似乎只会改善预填充阶段, 我们发现它也有助于降低解码时间, 因为它减少了每个实例所需的计算量, 从而使实例能够把更多 GPU 时间分配给解码阶段。

4.3 KVCACHE 感知与负载均衡: 二者的权衡

尽管这一思路直观, 但将 KVCACHE 感知真正整合进系统并不简单。根本原因在于, 考虑 KVCACHE 感知可能会干扰负载均衡。具体而言, 如前所述, 当我们使用线性组合定义分数时, 两个目标的优先级由打分函数中各指标的权重控制 (在图 6 (b) 中为 $\lambda = 0.4$): 如果给 KVCACHE 项分配更大的权重, 路由器就会优先把请求发往 KVCACHE 命中率更高的实例, 即便其他实例的负载明显更低。而这反过来又可能造成实例间的负载失衡。

图 11 通过展示不同 KVCACHE 感知权重下、上一节所述线性组合策略 (图 6 (b)) 的整体 TTFT 与 TPOT, 说明了这种权衡。我们可以看到, 当权重从 0.4 提升到 0.9 时, 除 API 轨迹外, TTFT 都是先下降后上升; API 轨迹受影响较小, 这是因为其输入长度较短。

为了揭示这种权衡背后的原因, 图 9 进一步展示了在 ChatBot 轨迹上改变 KVCACHE 感知权重时的 KVCACHE 命中率变化; 其他轨迹也表现出类似趋势。可以清楚地看到, 随着 KVCACHE 权重增加, 整体 KVCACHE 命中率也随之上升。这解释了为什么当权重从 0.4 增加到 0.7 时, TTFT 会下降。

尽管 KVCACHE 命中率提高了, 但在某个权重拐点之后 (例如 ChatBot 上的 0.7), 整体时延反而开始上升。这源于实例间的负载失衡; 我们在图 10 中刻画了这一现象。该图展示了在同一个 400 秒突发区间内, 使用两种不同线性组合权重 (0.7 vs. 0.9) 服务同一条 ChatBot 轨迹时, 两台实例所承担的预填充工作量。这里的预填充时间指每个 10 秒时间窗内, 该实例用于预填充的总秒数。这是衡量实例工作负载的一个良好指标, 因为直观上大多数 token 都是在解码阶段生成的; 因此, 如果某个实例

的处理时间被预填充主导, 那么它生成的 token 数通常就会少于其他实例。为便于展示, 我们从总共 16 台实例中选取预填充时间标准差最高的两台。我们观察到, 当 KVCACHE 优先级更高 ($\lambda = 0.9$) 时, 这两台实例的总预填充时间差异非常明显。相比之下, 在较为均衡的设置 ($\lambda = 0.7$) 下, 两台实例的预填充时间相近, 而且这一时期的平均预填充时间也相近。具体地说, 在 $\lambda = 0.7$ 时, 两台实例的平均预填充时间分别为 3.43s 和 3.40s; 而在 $\lambda = 0.9$ 时, 则分别为 3.57s 和 2.17s。

4.4 线性组合的情形

基于前面对权衡关系的分析, 通过线性组合同时实现 KVCACHE 感知和负载均衡, 会带来两个问题:

缺点 #1: 需要面向工作负载的超参数调优。 KVCACHE 的重要性及其对负载失衡的影响, 都依赖于具体工作负载。例如, 图 11 给出了在 Qwen3-30B 模型下, 不同轨迹的调参结果。可以看到, 不同工作负载的最优权重并不相同: 例如在 ChatBot 上最优权重是 0.7, 而在 API 上则是 0.55, 尽管它们的 KVCACHE 访问模式相近。需要注意的是, 我们无法负担遍历所有可能配置的代价, 因为在测试床上回放每一条轨迹都会消耗大量 GPU 时间。因此, 线性组合的性能依赖于面向具体工作负载的超参数调优, 而在实践中, 由于工作负载高度多样, 这种调优并不容易 [39]。

缺点 #2: 性能仍然可能次优。 在 §6 的评估中, 我们发现, 静态调好的权重并不总能取得与其他基线相竞争的性能。我们推测, 原因在于最优权重本身可能会随时间变化; 例如, 不同请求的 KVCACHE 命中模式就可能随时间变化 (见图 5 的第二行)。然而, 据我们所知, 现有所有工作都在整个服务期间使用一个固定的调优权重。虽然理论上可以设计随时间自适应调权的策略, 但这会进一步增加系统复杂度。

4.5 基于过滤组合的情形

方法。 图 13 展示了在 AIBrix [3] 等典型 LLM 调度系统中, 一种将 KVCACHE 感知与负载均衡结合起来的典型过滤式方法。首先, 路由器会检查当前集群是否负载失衡——即所有实例间最大批大小与最小批大小之差 ($BS.max() - BS.min()$) 是否超过某个阈值 (第 3 行)。如果超过, 路由器就放弃 KVCACHE 感知, 直接把请求路由到批大小最小的实例, 以恢复负载均衡 (第 4-5 行)。否则, 路由器使用 KVCACHE 命中率这一指标, 在实例之间进行选择 (第 6-9 行), 从而实现 KVCACHE 感知。

缺点 #1: 仍然需要面向工作负载的超参数调优。 与线性组合类似, 基于过滤的方法同样需要调参, 因为用于判断负载失衡的阈值 (Range) 依赖于具体工作负载。正如图 12 所示, 某个典型过滤方法 [3] 的最优阈值会随工作负载而变化: 例如在 Coder 轨迹上, 将阈值从 4 提升到 16, 可以分别改善 44% 的 P50 TTFT 和 15% 的 TPOT; 而在 API 轨迹上, 16 又明显优于 4。

缺点 #2: 性能仍然次优。 除了需要调参之外, 基于过滤的组合方法通常也比调到合适权重的线性组合更慢,

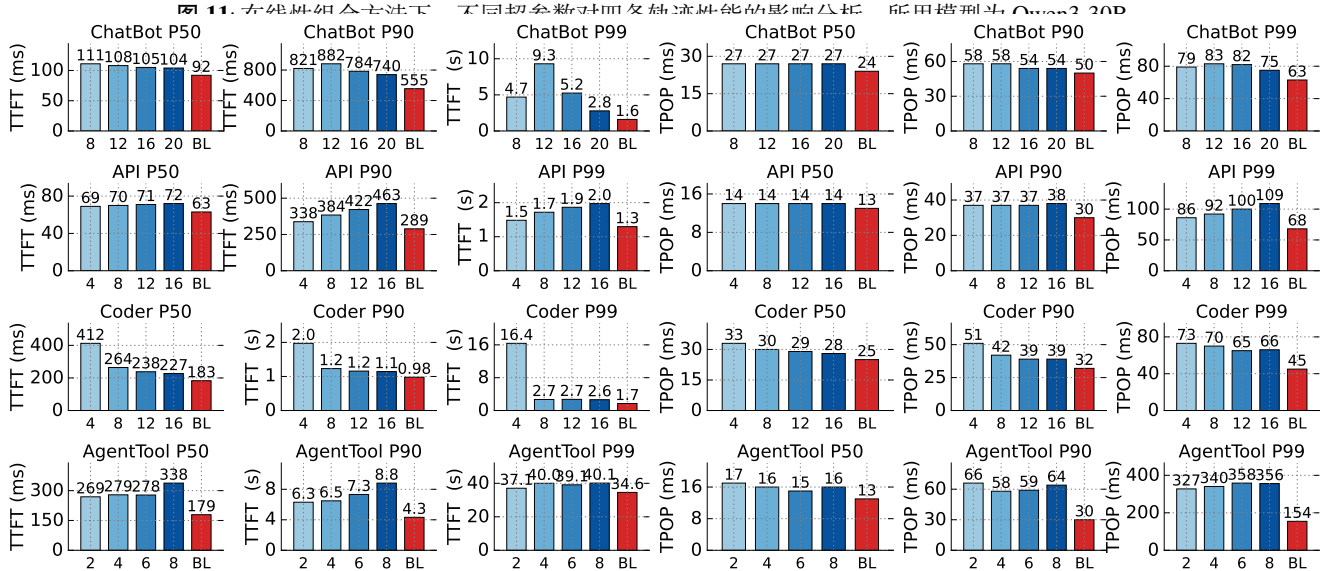
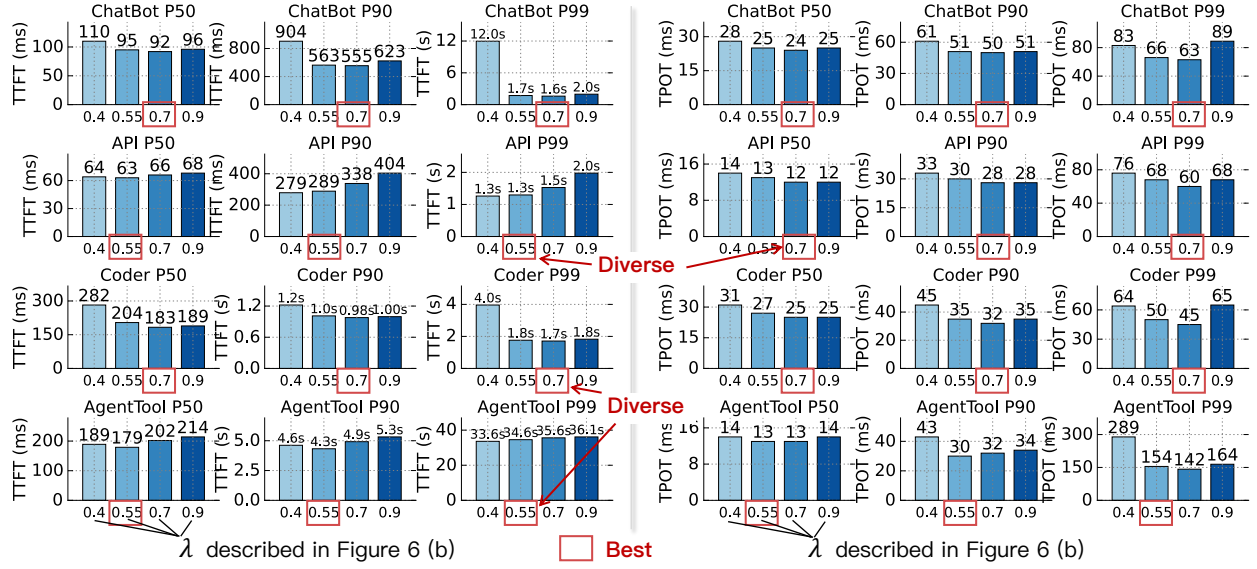


图 12: 在基于过滤的组合方法下, 不同超参数对四条轨迹性能的影响分析。BL 表示用于对比的线性组合方法, 其超参数已调到最优。图中的数字 (2,4,6,8) 对应图 13 中描述的范围。所用模型为 Qwen3-30B。

```

1 req = receive()
2 kv_hit = KV$.match(req)
3 if BS.max()- BS.min() > Range:
4     # Load balance
5     sched_to = instances
6     .select_min(BS)
7 else: # KV$-awareness
8     sched_to = instances
9     .select_max(kv_hit)
10    .select_min(bs)
11 req.forward(sched_to)

```

Filter-based method
(e.g., Aibrix-KV\$)

图 13: 在 LLM 调度中, 将 KVCache 感知与负载均衡结合的基于过滤方法伪代码, 由 Aibrix [3] 的 prefix-cache 策略简化而来。

如图 12 所示。这是因为它偏向负载均衡, 可能放弃了 KVCache 感知带来的收益。更具体地说, 一旦检测到

负载失衡, 它就会完全忽略 KVCache 感知; 但在某些情况下, 即使存在一定失衡, 把请求路由到具有更高 KVCache 命中率的实例仍可能是有益的, 因为显著减少的计算量本身也有助于缓解负载。在线性组合中, 只要分配给 KVCache 命中率的权重不太小, 这一点是可以做到的; 而基于过滤的方法无法表达这种折中。

4.6 基于模拟组合的情形

方法。最后, 基于模拟的方法 [32, 19, 47, 10] 会把“请求被路由到某个实例后, 其模拟服务时间”作为路由分数。这一思路基于这样一个观察: LLM 请求的执行过程高度结构化, 因此可以较准确地模拟。图 14 展示了一种典型模拟式方法 [19] 的伪代码: 在接收到请求后, 路由器先通过模拟器 (例如 VIDUR [1]) 估计请求路由到


```

1 req = receive()
2 TTFT = Simulatormodel.sim(req, BS, KV$, ...)
3 sched_to = instances.select_min(TTFT)
4 req.forward(sched_to)

```

图 14: 在 LLM 调度中, 将 KVCache 感知与负载均衡结合起来的基于模拟方法伪代码。

每个实例后的 TTFT (第 2 行), 然后把请求发送到估计 TTFT 最低的实例 (第 3–4 行)。需要注意的是, 这类方法通常只估计 TTFT, 而不是端到端时延, 因为端到端时延还取决于输出 token 数, 而这是不可预测的。

基于模拟的解决方案可以被看作是一种更高阶的指标组合方式, 因为为了准确模拟 TTFT, 模拟器必须把每个实例的 KVCache 状态和批大小作为输入特征。在这里, 我们选择了一个与 llm-d [19] 类似的模拟式实现, 但将其中的模拟器替换为改造后的 VIDUR [1]——这是当前最先进的 LLM 实例模拟器。我们的改造包括两部分: (1) 扩展模拟器, 使其在模拟预填充阶段时考虑带缓存命中的 KVCache 感知执行; (2) 用 Rust 重新实现它, 并启用并行模拟, 以便在线上支持跨多个实例的模拟调度。如果不考虑 KVCache 感知, 模拟式方案就无法超越其他方法, 这与我们在 §4.2 中的观察一致。如果不使用 Rust, 原始的 Python 实现会带来明显的调度延迟, 并不适合在线调度。我们还做了若干优化, 使模拟器能够扩展到数百个实例; 不过这些内容并非本文重点, 将留待后续论文展开。

我们研究了一种最先进的方法, 发现它在某些轨迹上优于线性组合方法, 这一点也在图 14 中有所体现。尽管性能有所提升, 基于模拟的方法由于其复杂性, 仍存在两个问题:

缺点 #1: 由于按模型开发和按硬件调优带来的开发复杂度。 基于模拟的方法性能高度依赖模拟器的准确性, 而要在实践中达到高准确度并不容易, 因为我们既要考虑模型架构, 又要考虑硬件特性。为了量化模拟器准确度对调度性能的影响, 图 15 给出了在 Qwen3-30B 模型上, 使用“调优良好”与“未调优”模拟器时四条轨迹的性能对比。这里, “未调优”模拟器原本是为另一种模型——Qwen2-7B——设计的; 而“调优良好”模拟器则是专门为 Qwen3-30B 实现的。图 16 进一步比较了这两个模拟器在服务 Qwen3-30B 时、相对 vLLM 的 TTFT 偏差。可以看到, 调优良好的模拟器准确度明显更高; 借助更准确的模拟器, TTFT 与 TPOT 的尾部时延可分别改善 75.6% 和 79.7%。

虽然可以为每种模型开发专门的模拟器, 以换取更高的调度性能, 但这样会带来不小的开发复杂度, 因为模拟过程高度依赖模型架构, 而模型架构又在快速演进, 不断出现新模块 (例如线性注意力 [33] 和 Engram [11])。与此同时, 即便是同一个模型, 模拟器的超参数也需要针对不同硬件特性重新调优, 进一步增加了开发成本。

缺点 #2: 性能仍然可能次优。 基于模拟的方法仍会遭遇次优性能问题, 尤其体现在 TPOT 上。如图 15 所示, 在 AgentTool 轨迹上, 它的 TPOT 尾部时延比表现最好的线性组合方法高 71.1%。我们推测, 这可能是因为模拟

器的一些误判导致了负载失衡。例如, 在图 16 中可以看到, 即便使用调优良好的模拟器, 仍有大约 10% 的请求预测误差超过 20%。这类误差主要来自两个方面: vLLM API 服务器中的请求重排序, 以及时延预测本身的不准确。

5 简单乘法或许就是所需的一切

方法概述 (图 17)。 我们的方法非常简单: 只需将两个精心选择的指标相乘来计算调度分数, 一个用于 KVCache 感知, 另一个用于负载均衡, 然后把请求路由到分数最小的实例。这一做法的基本思想是: 如果两个指标的线性组合有效, 那么乘法通常也能以类似方式工作, 同时还具备一个额外优势——无需进行超参数调优, 如图 (a) 所示。两个指标的具体选择——**P-token** (考虑 KVCache 命中后, 若将请求路由到某实例所需的新预填充 token 数) 以及 **BS** (实例的批大小)——来自我们在 §5.1 中的细致分析。

为了理解为什么把请求路由到 (P-Tokens × BS) 最小的实例能够同时兼顾两个目标, 考虑两个实例 i 和 j : 如果把请求路由到实例 i 相比路由到实例 j 能带来更多的 KVCache 命中, 那么实例 i 的 **P-token** 值就会更低, 除非实例 i 中已经排队了大量预填充请求 (这表明出现了工作失衡)。与此同时, **BS** 刻画了每个实例上的解码工作负载; 因此, 如果实例 i 的批大小显著大于实例 j , 乘法分数就很可能倾向于实例 j , 从而实现负载均衡。

我们也承认, 确实存在一些乘法可能失效的情形: 由于 KVCache 访问极度偏斜, 某一组实例会因为持续拥有很高的 KVCache 命中 (即很低的 **P-token**) 而总是被选中, 而这些实例上不断增大的批大小 (**BS**) 却不足以抵消如此低的 **P-token** 值。正如我们将在 §5.2 中分析的, 这类情形在实践中非常罕见, 并且可以事先检测到 (从而进行缓解)。

5.1 指标的选择

KVCache 感知指标 (P-token)。 除了 **P-token** 之外, 另一个自然的 KVCache 感知指标是 **1-KVCache 命中率**, 即请求被路由到某实例时的 KVCache 命中率。Preble [36] 和 AIGW [4] 等工作都采用了这种做法。这里取一减, 是因为更高的 KVCache 命中率应对应更低的分数, 从而与新增预填充 token 指标的方向保持一致。我们没有考虑 TTFT 作为指标, 因为它需要依赖模拟器的开发, 而这并不总是适用。

我们的经验分析表明, 使用 **P-token** 比使用 **1-KVCache 命中率** 能取得更好的性能。从图 18 (a) 可以看到, 在负载均衡指标都固定为 **BS** 的公平比较下, 使用 **P-token** 能够带来低 14.4% 的 P50 TTFT 以及低 42.8% 的 P95 TTFT。由于篇幅限制, 我们这里只报告一个工作负载的结果; 不过这一趋势在所有评估过的工作负载上都一致。

为了理解差异的根源, 图 18 在 (b) 中进一步拆解了不同方法的 KVCache 命中率, 并在 (c) 中展示了负载均衡状况。可以看到, 两种方法获得的 KVCache 命中率相近, 因此它们都以类似方式实现了 KVCache 感知。真正的

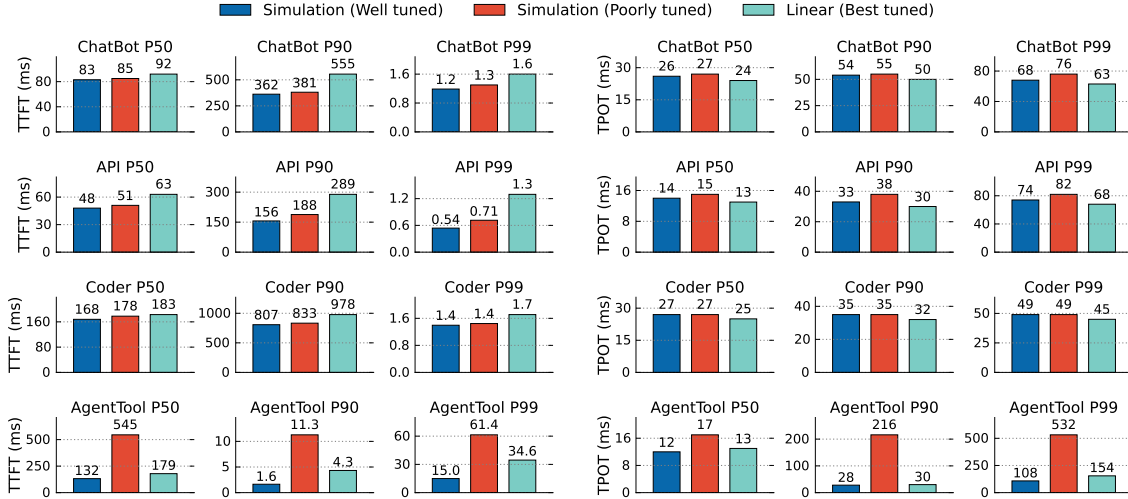


图 15: 在 Qwen3-30B 模型上, 不同模拟器准确度对四条轨迹性能影响的分析。

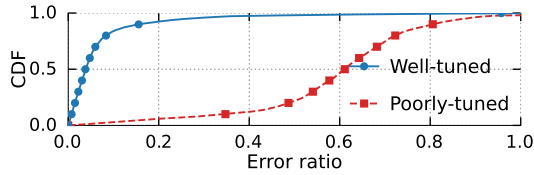


图 16: 在 Qwen3-30B 模型与 ChatBot 轨迹上, 调优良好的模拟器与未调优模拟器的对比。

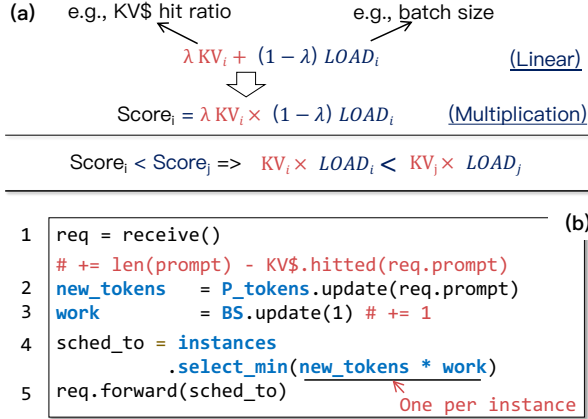


图 17: (a) 乘法如何在组合两个指标时避免线性组合所需超参数的示意图; 以及 (b) 我们调度方法的伪代码。

差异在于, 使用 **P-token** 能实现更好的负载均衡, 因为它额外考虑了每个实例上排队等待的预填充 token。因此, 在做调度决策时, 即便某些实例有较高的 KVCache 命中率, 路由器也会绕开那些已经排队了大量预填充请求的实例。

负载均衡指标 (BS)。除了 **BS** 外, 另一个常见选择是每个实例上的总 token 数 (**#Tokens**), ai-Dynamo [27]、AIGW [4] 等工作都采用了这一指标。其理由是, 一个请求的总计算成本与其上下文中的 token 数成正比。然而,

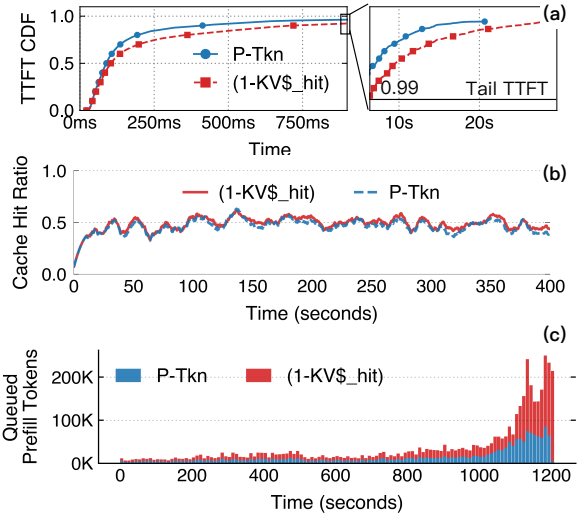


图 18: (a) 在 $A \times BS$ 调度中, 使用新增预填充 token (P-Tkn) 与使用 1-KVCache 命中率 (1-KV_{hit}) 作为 KVCache 感知指标 (A) 的比较; (b) KVCache 命中率分析; 以及 (c) 排队预填充 token 分析。该分析基于 Qwen3-30B 模型和 ChatBot(Qwen) 轨迹。

我们发现使用 **BS** 效果更好, 图 19 (a) 展示了这一点, 其原因有二。第一, 工作负载可以分为预填充负载与解码负载, 而前者已经由我们的 **P-Tokens** 指标刻画。因此, 我们只需要一个描述解码负载的指标, 而这正是 **BS** 所捕获的内容 (我们也尝试过使用 decode batch size, 结果类似, 因为 **BS** 主要由解码请求主导)。第二, **BS** 更适合作为实例上 (解码) 工作量的指标, 因为解码时间在不同批大小下更稳定: 更大的批大小往往对应更长的解码时间; 但更多的解码 token 并不一定意味着更长的解码时间, 因为在小批次情况下, 借助 KVCache, 解码可能依然很快 [46]。这一点在图 19 (b) 中得到了说明: 我们在那里刻画了在 Qwen3-30B 上服务 ChatBot(Qwen) 工作负载时, 批大小与总 token 数之间的关系。

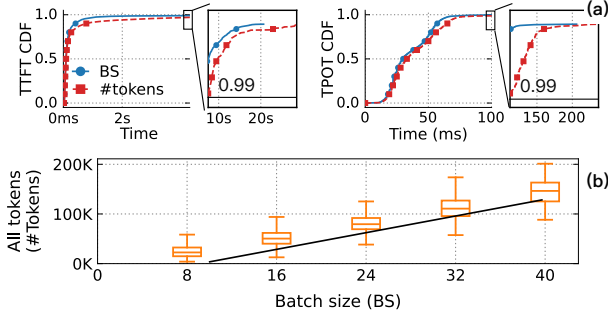


图 19: (a) 在 P -Tokens $\times B$ 调度中，使用批大小 (BS) 与使用总 token 数 (#Tokens) 作为负载均衡指标 (B) 的比较。(b) 对批大小与总 token 数关系的剖析。该分析基于 Qwen3-30B 模型和 ChatBot(Qwen) 轨迹。

5.2 基于乘法调度分数的良性情形与失效情形分析

概览。 从高层看，只要不存在负载失衡——也就是某些实例过载而其他实例空闲——那么把请求路由到具有较高 KVCache 命中率（即较低 **P-token**）的实例总是有利的。因此，当一组实例即将发生负载失衡，但这些实例上不断增加的 **BS** 无法抵消由于较高 KVCache 命中率而带来的 **P-token** 降低时，乘法就可能失效；结果就是请求会持续被路由到这些“即将过载”的实例上，并最终造成失衡。这类情况会出现在极端的 KVCache 偏斜下：某一小类请求反复访问一组实例上的同一前缀——我们将这种现象称为 **KVCache 热点**。我们发现，这种热点在实践中非常少见——至少在本文评估的所有轨迹中都没有出现；更重要的是，由于其模式非常清晰，我们可以设计探测器在事先将其识别出来。

因此，为了分析乘法方法的失效情形，我们需要推导出：在给定工作负载模式下，KVCache 热点何时会出现，以及在该情况下增加的批大小为何不足以抵消高 KVCache 命中率所带来的优势。对于每个工作负载，我们首先按 KVCache 前缀对请求分组，再依据 KVCache 命中率对实例做划分。随后，对每一个请求类别，我们都可以具体分析工作负载模式、由 KVCache 命中率决定的 **P-token** 指标，以及批大小指标三者之间的关系。

前奏：请求分组与实例划分。 首先，我们将所有请求划分为一组类别 C ，其中每个 $c \in C$ 对应一类共享相同 KVCache 前缀的请求。在实践中，一个类别大致对应某个应用或某个用户：它们的请求共享相同的系统提示词，并且往往拥有相似的对话历史 [39]。对一个固定类别 c ，我们考虑一个累积时间窗 $(t, t + window)$ ，并用 x 表示在该时间窗内到达集群的所有请求中属于类别 c 的比例；剩余比例则记作 $\bar{x} = 1 - x$ 。

对于每个类别 c ，我们再把集群划分为两组实例： M 和 $\bar{M}$ 。 M 包含当前缓存中保存了类别 c 前缀、即存在 KVCache 命中的实例；而 $\bar{M}$ 则包含所有其他实例。

请求类别对批大小的影响。 设 QPS 为集群总请求率，并令 t 表示可能导致失衡的类别 c 请求到达的时刻。我们记在时刻 t 之前、系统处于平衡状态时，每个实例的平均批大小为 BS_0 。同时，用 BS_t 和 $\bar{BS}_t$ 分别表示在区

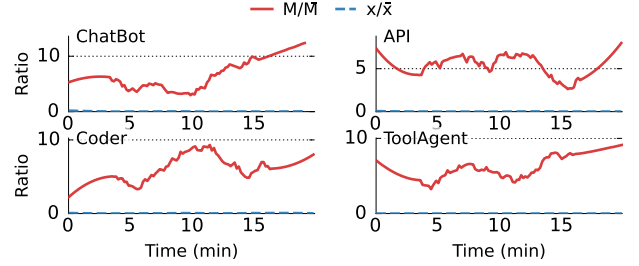


图 20: 不同轨迹中乘法分数相关因素的经验观测。如果 $\frac{x}{\bar{x}} \leq \frac{|M|}{|\bar{M}|}$ ，则不会出现由 KVCache 热点导致的负载失衡，因此我们的乘法方法能够在保持 KVCache 感知的同时有效实现负载均衡。

间 $(t, t + window)$ 内， M 与 $\bar{M}$ 中实例的期望批大小。我们考虑一种极端情况：由于极高的 KVCache 命中率，所有属于类别 c 的请求都会被路由到 M ；因为如果其中一部分请求被路由到 $\bar{M}$ ，那么热点本身就已经通过把流量分散到更多机器上而得到缓解。因此，我们可以得到一个近似表达式，用来刻画“潜在过载实例”的批大小与“非过载实例”的批大小之比：

$$\frac{BS_t}{\bar{BS}_t} = \frac{BS_0 + (x \cdot QPS) / |M| \cdot t}{BS_0 + (\bar{x} \cdot QPS) / |\bar{M}| \cdot t}. \quad (1)$$

主导项 $(x \cdot QPS) / |M| \cdot t$ 对应的是类别 c 的请求被路由到缓存了其前缀的实例上的数量；而 $(\bar{x} \cdot QPS) / |\bar{M}| \cdot t$ 则对应其余所有请求被路由到剩余实例上的情况。

分析：良性情形，以及它们是否常见？ 只要疑似热点实例的批大小没有高于其他实例，那么把请求路由到这些实例显然是有益的，因为这种决策能够提升 KVCache 命中率，同时不会造成负载失衡。而我们的乘法分数正会忠实地做出这样的选择，因此这些情形对我们的方法而言属于“良性情形”。剩下的问题是：这种情形在实践中常见吗？幸运的是，基于方程 1 所建立的“工作负载—批大小”关系，我们可以通过追踪不同轨迹中的两个量 $\frac{x}{\bar{x}}$ 和 $\frac{|M|}{|\bar{M}|}$ ，对其出现频率进行经验分析。这里我们在每个时间窗（1 分钟）中采样 KVCache 命中率最高的请求类别。

$$\frac{x}{\bar{x}} \leq \frac{|M|}{|\bar{M}|} \quad (2)$$

图 20 表明，本文研究的所有轨迹——它们能够代表真实 LLM 服务场景——对于我们的方法都属于良性情形，因为疑似热点实例的期望批大小并不大于其他实例。这一点可以作如下解释。根据图 20，对所有轨迹我们都能观察到由方程 2 概括的关系。

$$\frac{x}{|M|} \leq \frac{\bar{x}}{|\bar{M}|} \quad (3)$$

对方程进行整理后即可得到方程 3。将其代入方程 1，我们便得到：KVCache 热点实例的期望批大小小于其他实例。

分析：失效情形，以及我们的两阶段探测器。根据前述分析可知，如果方程 2 不成立，即

$$\frac{x}{\bar{x}} > \frac{|M|}{|\bar{M}|}$$

，那么就存在 KVCache 热点导致负载失衡、从而破坏我们乘法方法的可能性。这为失效情形提供了一个仅依赖工作负载模式与 KVCache 状态的清晰探测信号，因此我们可以在真正做出调度决策前，就开发一个探测器去捕获这类潜在失效。

具体而言，对于每个请求类别，我们实时统计两个量 $\frac{x}{\bar{x}}$ 与 $\frac{|M|}{|\bar{M}|}$ 。如果观察到方程 2 不成立，我们就触发第一阶段告警，并把疑似实例 (M) 从候选路由目标中临时过滤掉。为了避免追踪过多请求类别带来的开销，我们只跟踪那些 KVCache 命中率最高的请求。

单阶段探测器的问题在于：即使方程 2 不成立，把请求路由到拥有高 KVCache 命中的疑似实例，有时仍然是值得的，只要我们会把过多请求连续发往这些热点实例。因此，我们进一步加入第二阶段：并不在第一阶段告警时立即过滤，而是延迟到我们观察到某个类别的请求连续多次被路由到热点实例之后再采取行动；因为连续的路由决策往往意味着，疑似实例上批大小的增加已经不足以抵消其高 KVCache 命中率所带来的优势。

$$P\text{-token}_m \times BS_m \leq P\text{-token}_{\bar{m}} \times BS_{\bar{m}} \quad (4)$$

具体做法是：在第一阶段触发告警后，我们继续跟踪该类别请求的新增预填充 token；如果我们观察到，超过 $2 \times M$ 个连续请求在某个疑似热点实例 (m) 上的乘法分数（方程 4）都小于其在其他实例 ($\bar{m}$) 上的分数，那么就触发第二阶段告警，并把这些疑似实例从路由目标中过滤掉。

6 端到端评估

在对高效 LLM 调度的研究收尾时，我们通过端到端 LLM 服务性能，比较 LMetric 的乘法方法（见 §5）与当前最先进的调度器。

基线方法。 我们将 LMetric 与开源调度器以及 Bailian 当前生产环境所使用的调度器进行比较。由于一些实现会因为路由器本身的问题表现不佳，例如 vLLM 的 Python 路由器就存在吞吐偏低的问题 [38]，我们将这些策略重新实现在 §3 所述的高性能 Rust 路由框架中。为了做到真正可比的横向比较，我们在统一框架下比较这些策略，并仔细验证了我们的重实现不会慢于原始实现。各基线的详细说明如下：

- **Bailian** 是 Bailian 的 LLM 服务系统中正在使用的生产级调度器。它采用了一种与图 6 (b) 所示类似的线性组合方法。我们针对每个工作负载都仔细调节了其超参数，以获得最佳性能。
- **vLLM [37]** 是一个被广泛使用的 LLM 服务系统，采用图 6 (a) 所描述的“仅负载均衡”设计。

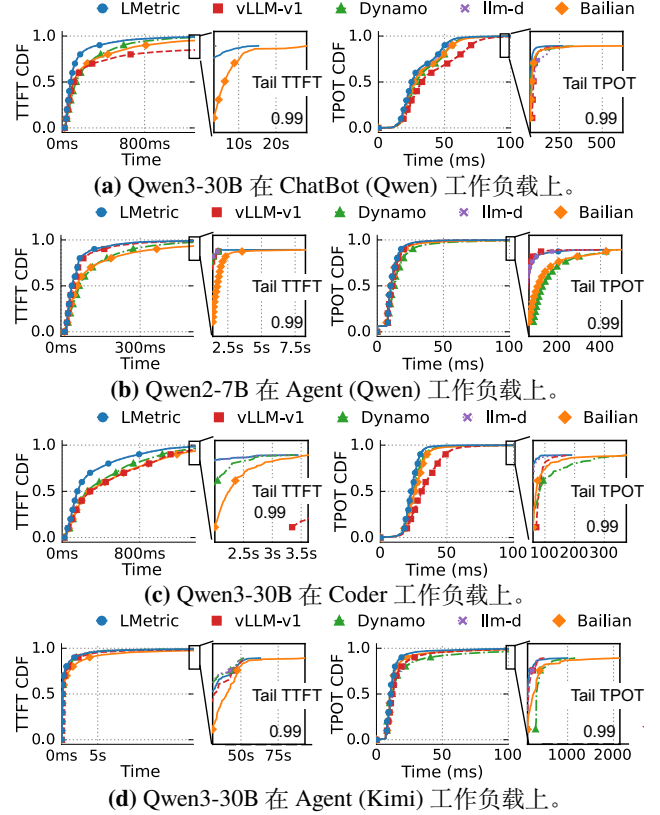


图 21: LMetric 与各基线在四个工作负载上的端到端 TTFT 与 TPOT CDF。

- **Dynamo [27]** 是 NVIDIA 发布的流行服务框架。它同样采用基于线性组合的方式，但所选指标与 Bailian 不同。其用于 KVCache 感知的指标是预填充 token 数（与我们的 P-token 相同），用于负载均衡的指标则是实例上的总 token 数。路由器会将请求发送到这两个指标经归一化并加权求和后最小的实例。与 Bailian 类似，我们也为其在每个工作负载上调节超参数，以获得最优性能。
- **llm-d [19]** 采用基于时延的调度策略：它会估计 TTFT，并依据 §4.6 所述的模拟方法，把请求路由到估计 TTFT 最低的实例。

整体性能。 图 21 展示了 LMetric 与其他基线在四条轨迹上的端到端 TTFT 与 TPOT CDF。所有实验都在一套 16 GPU、16 实例的测试床上进行，并将轨迹缩放到测试床可承载最大负载的一半。由于篇幅限制，我们只报告具有代表性的结果子集：Qwen3 MoE 模型在 ChatBot、Coder 和 Agent 工作负载上的结果，以及 Qwen2 模型在 API 工作负载上的结果。我们观察到，在所有“模型—轨迹”组合上，性能趋势都保持一致。

LMetric 在所有轨迹上都优于全部基线。在 ChatBot 工作负载上，与 vLLM 相比，它将平均 TTFT 降低了 92%，将平均 TPOT 降低了 24%；与 llm-d——第二好的策略——相比，它在 TTFT 近似的情况下，将 P99 TPOT 降低了 13%。性能提升的主要原因在于：它具备 KVCache 感知能力，同时又没有牺牲负载均衡。为了进一步理解

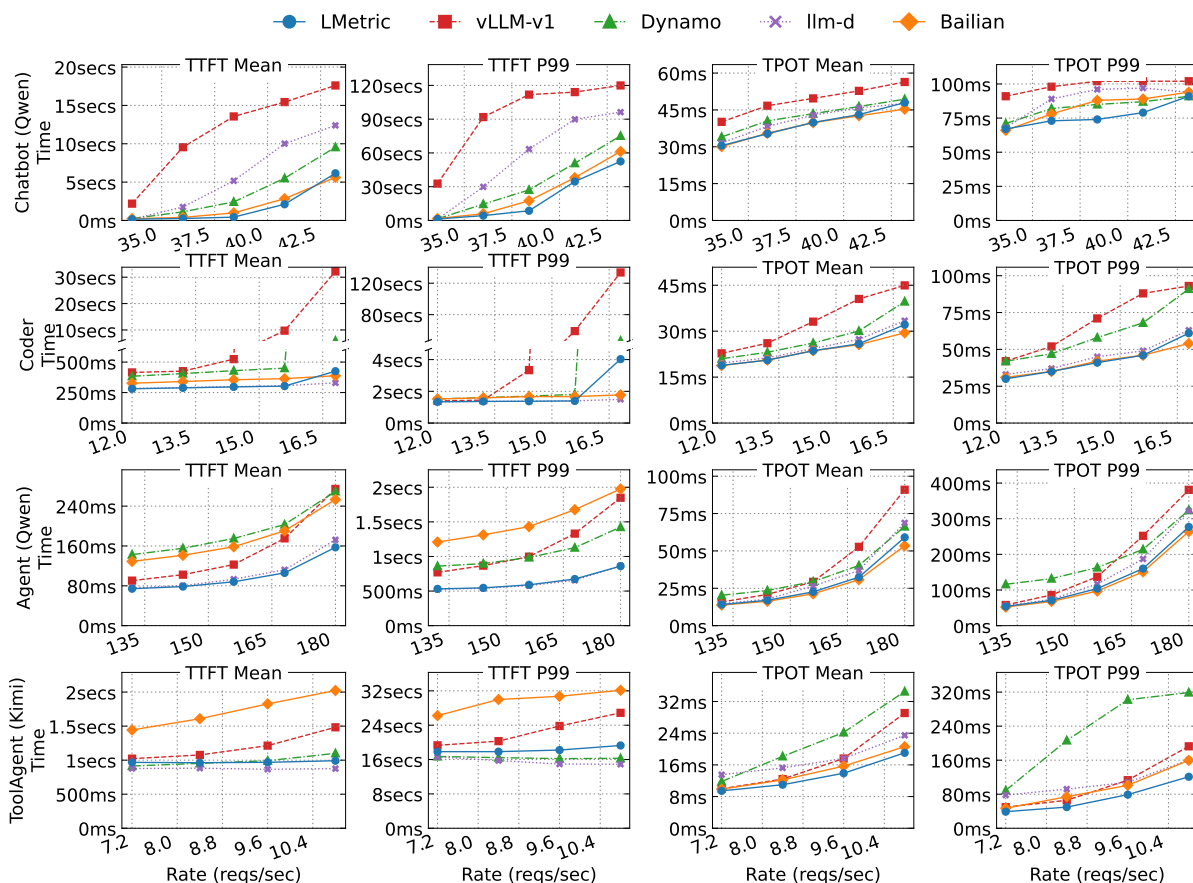


图 22: 不同请求速率下的端到端性能。除第二行使用 Qwen2-7B 模型外, 其余各行均使用 Qwen3-30B 模型。

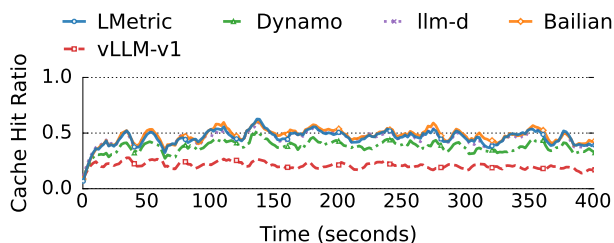


图 23: Qwen3-30B 模型在 ChatBot (Qwen) 工作负载上, 各策略的 KVCache 命中率对比。

LMetric 的行为, 图 23 绘制了在 ChatBot 工作负载上, 不同系统的 KVCache 命中率。可以看到, LMetric 始终能够获得与其他 KVCache 感知策略相当的 KVCache 命中率, 并且显著高于不感知 KVCache 的策略 (vLLM)。与此同时, 图 24 进一步沿用 §4.3 的分析方法, 对负载失衡进行了剖析。为了便于展示, 我们只比较 LMetric 与 llm-d——后者是在 ChatBot 轨迹上表现第二好的方法。可以看到, 相比 llm-d, LMetric 实现了更均衡的负载。

不同请求速率下的性能。 图 22 进一步展示了 LMetric 在不同请求到达率下的表现。结果与固定请求速率设置下基本一致: 在不同轨迹和请求速率上, LMetric 都优于其他基线; 唯一的例外是 ToolAgent 轨迹, 在该轨迹

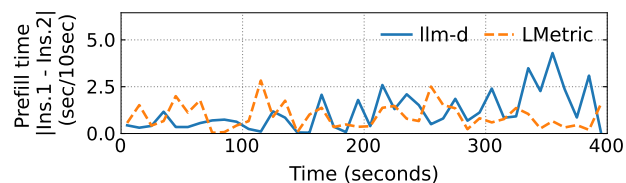


图 24: LMetric 与 llm-d 在服务 ChatBot (Qwen) 工作负载、使用 Qwen3-30B 模型时, 两台实例之间工作负载失衡情况的剖面图。图中报告的指标是每个 10 秒时间窗内, 两台实例 (Inst.) 已服务预填充时间的绝对差值。

上, LMetric 的平均 TTFT 比 llm-d 略高 (10%), 但 TPOT 降低了 30%。这可能是因为基于模拟的方法能够比我们简单的 P-token 指标更好地估计预填充工作量。尽管如此, LMetric 依然在不依赖复杂模拟器的前提下取得了最低的 TPOT, 因为它同时考虑了 KVCache 管理与 (解码) 负载均衡。随着请求速率提高, 不同基线之间的性能差距会进一步拉大, 因为在高负载下, 更均衡且更具 KVCache 感知能力的调度策略能够更有效地提升系统整体吞吐, 从而更快地消化队列中的请求。

7 相关工作

LLM 请求的全局调度。 LMetric 延续了在集群中调度 LLM 请求这一研究方向 [23, 21, 36, 32, 47, 19]。据我们所知, 这些方法都试图通过三类方式组合不同指标, 从而同时实现 KVCache 感知和负载均衡; 我们已在 §4 中对此进行了详细讨论。LMetric 建立在这些工作的基础上, 复用了它们所使用的指标, 但进一步提出了一种新的、简单的乘法组合器; 大量评估结果也验证了这种方法的有效性。

实例内部的 LLM 请求调度。 除了跨实例的全局调度之外, 还有一类工作关注实例内部的局部调度优化。Sarathi-Serve [2] 提出了 chunked prefill, 将长预填充请求切分为更小的块, 以减少与之同置的解码请求的停顿时间。VTC [35] 采用基于 token 的准入控制机制来实现公平性。FairBatching [24] 使用线性时间分析模型, 在预填充与解码 token 之间进行优先级决策, 并动态构造批次。这些工作与我们的全局调度是正交的; 而一个好的全局调度器还能进一步增强局部调度的效果, 例如减少“实例过载”这类仅靠局部调度难以解决的问题发生的频率。

LLM 服务优化。 除调度之外, 研究界还提出了多种提升 LLM 服务性能的技术。其中包括提升 KVCache 命中率的方法 [32, 39]、为模型服务提供弹性的机制 [44, 42], 以及优化 GPU 执行效率的方法 [23, 22], 等等。据我们所知, 这些技术都可以与调度优化共存, 因此也能够与 LMetric 结合, 进一步提升整体性能。

8 结论

本文提出了首个基于乘法组合器的高质量 LLM 请求调度方法, 能够在无须超参数调优的情况下, 同时实现 KVCache 感知与负载均衡。在覆盖聊天机器人、API 调用和代码智能体的真实工作负载上的评测表明, 我们的方法相较于 vLLM [37]、ai-Dynamo [27]、llm-d [19] 以及 Bailian 生产环境中使用的调度器都更具优势。

参考文献

- [1] AGRAWAL, A., KEDIA, N., MOHAN, J., PANWAR, A., KWATRA, N., GULAVANI, B. S., RAMJEE, R., AND TUMANOV, A. VIDUR: A large-scale simulation framework for LLM inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024* (2024), P. B. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., mlsys.org.
- [2] AGRAWAL, A., KEDIA, N., PANWAR, A., MOHAN, J., KWATRA, N., GULAVANI, B., TUMANOV, A., AND RAMJEE, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 117–134.
- [3] Aibrix. <https://github.com/vllm-project/aibrix>, 2025.
- [4] Aigw. <https://github.com/aigw-project/aigw>, 2025.
- [5] ALI, A., PINCIROLI, R., YAN, F., AND SMIRNI, E. Optimizing inference serving on serverless platforms. *Proc. VLDB Endow.* 15, 10 (2022), 2071–2084.
- [6] ANTHROPIC. Claude api. <https://www.anthropic.com/api>, 2025.
- [7] ARAPAKIS, I., BAI, X., AND CAMBAZOGLU, B. B. Impact of response latency on user behavior in web search. In *The 37th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '14, Gold Coast, QLD, Australia - July 06 - 11, 2014* (2014), S. Geva, A. Trotman, P. Bruza, C. L. A. Clarke, and K. Järvelin, Eds., ACM, pp. 103–112.
- [8] AZURE. Azure llm inference traces. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2024.md>, 2024.
- [9] Qwen-bailian anonymous dataset. <https://github.com/alibaba-edu/qwen-bailian-usagetraces-anon>, 2025.
- [10] CHEN, S., JIA, Z., KHAN, S., KRISHNAMURTHY, A., AND GIBBONS, P. B. Slos-serve: Optimized serving of multi-slo llms. *CoRR abs/2504.08784* (2025).
- [11] CHENG, X., ZENG, W., DAI, D., CHEN, Q., WANG, B., XIE, Z., HUANG, K., YU, X., HAO, Z., LI, Y., ZHANG, H., ZHANG, H., ZHAO, D., AND LIANG, W. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026.
- [12] DELIMITROU, C., AND KOZYRAKIS, C. Quasar: resource-efficient and qos-aware cluster management. In *Architectural Support for Programming Languages and Operating Systems, ASPLOS 2014, Salt Lake City, UT, USA, March 1-5, 2014* (2014), R. Balasubramanian, A. Davis, and S. V. Adve, Eds., ACM, pp. 127–144.
- [13] FlashInfer: Kernel Library for LLM Serving. <https://github.com/flashinfer-ai/flashinfer>, 2025.
- [14] GAO, B., HE, Z., SHARMA, P., KANG, Q., JEVDIJC, D., DENG, J., YANG, X., YU, Z., AND ZUO, P. Cost-Efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)* (Santa Clara, CA, July 2024), USENIX Association, pp. 111–126.
- [15] GIGASPACE. Amazon found every 100ms of latency cost them 1% in sales. <https://www.gigaspaces.com/blog/amazon-found-every-100ms-of-latency-cost-them-1-in-sales>, 2024.
- [16] GITHUB. Accelerate your development speed with copilot. <https://copilot.github.com>, 2024.
- [17] GOG, I., SCHWARZKOPF, M., GLEAVE, A., WATSON, R. N. M., AND HAND, S. Firmament: Fast, centralized cluster scheduling at scale. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016* (2016), K. Keeton and T. Roscoe, Eds., USENIX Association, pp. 99–115.

- [18] GOOGLE. Gemini api. <https://ai.google.dev/api>, 2025.
- [19] GOOGLE. llm-d. <https://github.com/llm-d/llm-d>, 2025.
- [20] GUJARATI, A., KARIMI, R., ALZAYAT, S., HAO, W., KAUFMANN, A., VIGFUSSON, Y., AND MACE, J. Serving dnns like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020* (2020), USENIX Association, pp. 443–462.
- [21] HU, X., ZENG, T., YUAN, X., SONG, L., ZHANG, G., AND HE, B. Bestserve: Serving strategies with optimal goodput in collocation and disaggregation architectures. *CoRR abs/2506.05871* (2025).
- [22] KAMATH, A. K., PRABHU, R., MOHAN, J., PETER, S., RAMJEE, R., AND PANWAR, A. Pod-attention: Unlocking full prefill-decode overlap for faster LLM inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025* (2025), L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds., ACM, pp. 897–912.
- [23] KWON, W., LI, Z., ZHUANG, S., SHENG, Y., ZHENG, L., YU, C. H., GONZALEZ, J., ZHANG, H., AND STOICA, I. Efficient memory management for large language model serving with page-dattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023* (2023), J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds., ACM, pp. 611–626.
- [24] LYU, H., LIU, B., WU, M., AND CHEN, H. Fairbatching: Fairness-aware batch formation for LLM inference. *CoRR abs/2510.14392* (2025).
- [25] MIAO, X., SHI, C., DUAN, J., XI, X., LIN, D., CUI, B., AND JIA, Z. Spotserve: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024* (2024), R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafir, Eds., ACM, pp. 1112–1127.
- [26] Mooncake trace. https://github.com/kvccache-ai/Mooncake/blob/main/FAST25-release/traces/toolagent_trace.jsonl, 2025.
- [27] NVIDIA. ai-dynamo. <https://github.com/ai-dynamo/dynamo>, 2025.
- [28] OPENAI. Openai developer platform. <https://platform.openai.com/docs/overview>.
- [29] OPENAI. Chatgpt. <https://chatgpt.com>, 2025.
- [30] PATEL, P., CHOUKSE, E., ZHANG, C., SHAH, A., GOIRI, Í., MALEKI, S., AND BIANCHINI, R. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024* (2024), IEEE, pp. 118–132.
- [31] Pollaczek–khinchine formula. https://en.wikipedia.org/wiki/Pollaczek-Khinchine_formula#cite_note-2, 2025.
- [32] QIN, R., LI, Z., HE, W., CUI, J., REN, F., ZHANG, M., WU, Y., ZHENG, W., AND XU, X. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)* (Santa Clara, CA, Feb. 2025), USENIX Association, pp. 155–170.
- [33] Qwen3-next. <https://qwen.ai/blog?id=4074cca80393150c248e508aa62983f9cb7d27cd&from=research.latest-advancements-list>, 2026.
- [34] SAJAL, S. M., ZHU, T., URGONKAR, B., AND SEN, S. Traceup-scaler: Upscaling traces to evaluate systems at high load. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024* (2024), ACM, pp. 942–961.
- [35] SHENG, Y., CAO, S., LI, D., ZHU, B., LI, Z., ZHUO, D., GONZALEZ, J. E., AND STOICA, I. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 965–988.
- [36] SRIVATSA, V., HE, Z., ABHYANKAR, R., LI, D., AND ZHANG, Y. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025* (2025), OpenReview.net.
- [37] vllm v0.12.0 release. <https://github.com/vllm-project/vllm/releases/tag/v0.12.0>, 2025.
- [38] [bugfix]: Avoid unnecessary coordination for non-moe data parallel. <https://github.com/vllm-project/vllm/issues/24461>, 2026.
- [39] WANG, J., HAN, J., WEI, X., SHEN, S., ZHANG, D., FANG, C., CHEN, R., YU, W., AND CHEN, H. Kvcache cache in the wild: characterizing and optimizing kvcache cache at a large cloud provider. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (USA, 2025), USENIX ATC '25, USENIX Association.
- [40] WANG, J., HAN, J., WEI, X., SHEN, S., ZHANG, D., FANG, C., CHEN, R., YU, W., AND CHEN, H. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)* (July 2025), USENIX Association.
- [41] WANG, Y., CHEN, Y., LI, Z., KANG, X., FANG, Y., ZHOU, Y., ZHENG, Y., TANG, Z., HE, X., GUO, R., ET AL. Burstgpt: A real-world workload dataset to optimize llm serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2* (2025), pp. 5831–5841.

- [42] XIANG, Y., LI, X., QIAN, K., YANG, Y., ZHU, D., YU, W., ZHAI, E., LIU, X., JIN, X., AND ZHOU, J. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP 2025, Lotte Hotel World, Seoul, Republic of Korea, October 13-16, 2025* (2025), Y. Won, Y. Kwon, D. Yuan, and R. Isaacs, Eds., ACM, pp. 1030–1045.
- [43] XIANG, Y., LI, X., QIAN, K., YU, W., ZHAI, E., AND JIN, X. Servegen: Workload characterization and generation of large language model serving in production. *CoRR abs/2505.09999* (2025).
- [44] ZHANG, D., WANG, H., LIU, Y., WEI, X., SHAN, Y., CHEN, R., AND CHEN, H. Blitzscale: Fast and live large model autoscaling with O(1) host caching. In *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025* (2025), L. Zhou and Y. Zhou, Eds., USENIX Association, pp. 275–293.
- [45] ZHANG, X., TUNE, E., HAGMANN, R., JNAGAL, R., GOKHALE, V., AND WILKES, J. Cpi²: CPU performance isolation for shared compute clusters. In *Eighth Eurosys Conference 2013, EuroSys '13, Prague, Czech Republic, April 14-17, 2013* (2013), Z. Hanzálek, H. Härtig, M. Castro, and M. F. Kaashoek, Eds., ACM, pp. 379–391.
- [46] ZHONG, Y., LIU, S., CHEN, J., HU, J., ZHU, Y., LIU, X., JIN, X., AND ZHANG, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024* (2024), A. Gavrilovska and D. B. Terry, Eds., USENIX Association, pp. 193–210.
- [47] ZHU, K., SHI, H., XU, L., SHAN, J., KRISHNAMURTHY, A., KASIKCI, B., AND XIE, L. Polyserve: Efficient multi-slo serving at scale. *CoRR abs/2507.17769* (2025).